

Guia de Estudos: Sistemas Operacionais II

Instituto Federal Sul-rio-grandense - Câmpus Passo Fundo

Bacharelado em Ciência da Computação

Disciplina: Sistemas Operacionais II

Professor: Prof. Leonardo Deliyannis Constantin

Tópico: Problemas Clássicos de Sincronização e Concorrência

1. Fundamentação Teórica e Primitivas

A execução concorrente exige mecanismos para evitar que múltiplos processos acessem simultaneamente recursos compartilhados de forma inconsistente.

- **Condição de Corrida (Race Condition):** Situação onde o resultado final depende da ordem de execução das instruções no tempo.
- **Região Crítica:** Trecho de código que acessa recursos compartilhados. Apenas um processo pode estar nela por vez (**Exclusão Mútua**).
- **Semáforos:** Primitiva variável inteira acessada via:
 - **wait (P / sem_wait):** Decrementa o semáforo. Se < 0 , o processo bloqueia.
 - **signal (V / sem_post):** Incrementa o semáforo. Se houver processos bloqueados, um é acordado.
- **Condições de Coffman (Deadlock):** Para que ocorra um impasse total, quatro condições devem ser simultâneas:
 - a. **Exclusão Mútua:** Recurso não compartilhável.
 - b. **Posse e Espera:** Processo retém um recurso enquanto espera outro.
 - c. **Não Preempção:** Recursos não podem ser retirados à força.
 - d. **Espera Circular:** Cadeia fechada de processos esperando recursos uns dos outros.

2. Problema 1: Produtor-Consumidor (Buffer Limitado)

O desafio consiste em coordenar dois processos que compartilham um buffer de tamanho fixo N . O produtor não pode inserir em um buffer cheio (**overflow**) e o consumidor não pode retirar de um buffer vazio (underflow).

Solução com Semáforos

Utilizam-se três semáforos:

- **mutex**: Garante exclusão mútua no acesso ao buffer (inicializado em 1).
- **empty**: Conta espaços vazios (inicializado em N).
- **full**: Conta itens disponíveis (inicializado em 0).

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>
```

```
#define N 5          // Tamanho do buffer compartilhado
```

```
int buffer[N];
int in = 0;          // Índice de inserção do produtor
int out = 0;         // Índice de remoção do consumidor
```

```
sem_t mutex;        // Controla acesso à região crítica
sem_t empty;        // Conta posições livres (inicia em N)
sem_t full;          // Conta posições preenchidas (inicia em 0)
```

```
void* produtor(void* arg) {
    int item = 1;
    while (1) {
        sleep(1);    // Simula produção de item

        sem_wait(&empty); // Decrementa espaço livre
        sem_wait(&mutex);  // Entra na região crítica
```

```

    buffer[in] = item;
    printf("[PRODUTOR] Inseriu item %d no indice %d\n", item, in);
    in = (in + 1) % N;
    item++;

    sem_post(&mutex); // Sai da região crítica
    sem_post(&full);  // Notifica item disponível
}
return NULL;
}

void* consumidor(void* arg) {
    while (1) {
        sem_wait(&full); // Espera haver item disponível
        sem_wait(&mutex); // Entra na região crítica

        int item = buffer[out];
        printf(" [CONSUMIDOR] Retirou item %d do indice %d\n", item, out);
        out = (out + 1) % N;

        sem_post(&mutex); // Sai da região crítica
        sem_post(&empty); // Notifica vaga livre

        sleep(2); // Simula processamento do item consumido
    }
    return NULL;
}

int main() {
    pthread_t th_prod, th_cons;

    sem_init(&mutex, 0, 1);
    sem_init(&empty, 0, N);
    sem_init(&full, 0, 0);

    pthread_create(&th_prod, NULL, produtor, NULL);
    pthread_create(&th_cons, NULL, consumidor, NULL);

```

```
pthread_join(th_prod, NULL);
pthread_join(th_cons, NULL);

return 0;
}
```

3. Problema 2: Jantar dos Filósofos

Cinco filósofos sentam-se à mesa para pensar e comer. Para comer, precisam de dois garfos (esquerdo e direito). O problema clássico ilustra o risco de deadlock e starvation.

Estratégias de Resolução

1. **Assimetria:** Um filósofo (ex: o de índice ímpar) pega primeiro o garfo esquerdo e depois o direito, enquanto o de índice par faz o inverso. Isso quebra a **Espera Circular**.
2. **Solução de Tanenbaum (Estados):** Define estados **PENSANDO**, **FOME** e **COMENDO**. Um filósofo só começa a comer se ambos os vizinhos não estiverem comendo.
3. **Árbitro/Limitador:** Permitir que apenas $N-1$ filósofos entrem na sala de jantar simultaneamente, garantindo que sempre haverá ao menos um filósofo com dois garfos disponíveis.

```
include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5          // Número de filósofos
#define ESQUERDA (i + N - 1) % N // Vizinho da esquerda
#define DIREITA (i + 1) % N      // Vizinho da direita

#define PENSANDO 0
#define FOME     1
#define COMENDO  2
```

```

int estado[N];
sem_t mutex;           // Exclusão mútua para as regiões críticas
sem_t s[N];            // Um semáforo por filósofo (inicializado em 0)

void testar(int i) {
    if (estado[i] == FOME && estado[ESQUERDA] != COMENDO &&
        estado[DIREITA] != COMENDO) {
        estado[i] = COMENDO;
        printf("Filosofo %d pegou os garfos %d e %d e começou a COMER.\n", i,
            ESQUERDA, i);
        sem_post(&s[i]);    // Acorda o filósofo se ele estava bloqueado
    }
}

void pegar_garfos(int i) {
    sem_wait(&mutex);
    estado[i] = FOME;
    printf("Filosofo %d esta com FOME.\n", i);
    testar(i);            // Tenta pegar ambos os garfos
    sem_post(&mutex);
    sem_wait(&s[i]);        // Bloqueia caso os vizinhos estejam comendo
}

void soltar_garfos(int i) {
    sem_wait(&mutex);
    estado[i] = PENSANDO;
    printf("Filosofo %d soltou os garfos e voltou a PENSAR.\n", i);
    testar(ESQUERDA);     // Dá a chance para o vizinho da esquerda comer
    testar(DIREITA);      // Dá a chance para o vizinho da direita comer
    sem_post(&mutex);
}

void* filosofo(void* num) {
    int i = *(int*)num;
    while (1) {
        sleep(1);         // Pensando...
        pegar_garfos(i);
    }
}

```

```

        sleep(2);          // Comendo...
        soltar_garfos(i);
    }
    return NULL;
}

int main() {
    pthread_t thread_id[N];
    int ids[N];

    sem_init(&mutex, 0, 1);
    for (int i = 0; i < N; i++) {
        sem_init(&s[i], 0, 0);
        ids[i] = i;
    }

    for (int i = 0; i < N; i++) {
        pthread_create(&thread_id[i], NULL, filosofo, &ids[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(thread_id[i], NULL);
    }

    return 0;
}

```

4. Problema 3: Barbeiro Adormecido

Um barbeiro trabalha em uma barbearia com uma cadeira de corte e N cadeiras de espera. Se não há clientes, o barbeiro dorme. Se um cliente chega e o barbeiro está dormindo, ele o acorda. Se o barbeiro está ocupado, o cliente senta se houver cadeira livre; caso contrário, vai embora (desistência).

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

```

```

#include <semaphore.h>

#define CADEIRAS 3      // Cadeiras de espera na ante-sala

sem_t mutex;           // Protege a variável compartilhada 'num_espera'
sem_t clientes_esperando; // Barbeiro dorme aqui (inicia em 0)
sem_t barbeiro_pronto;  // Cliente espera a cadeira de corte aqui (inicia em 0)

int num_espera = 0;     // Clientes atualmente aguardando

void* thread_barbeiro(void* arg) {
    while (1) {
        sem_wait(&clientes_esperando); // Dorme até um cliente chegar

        sem_wait(&mutex);
        num_espera--;                // Retira o cliente da cadeira de espera
        sem_post(&barbeiro_pronto);  // Convida cliente para a cadeira principal
        sem_post(&mutex);

        printf("[BARBEIRO] Cortando cabelo...\n");
        sleep(3);                    // Executa o corte
        printf("[BARBEIRO] Corte concluído!\n");
    }
    return NULL;
}

void* thread_cliente(void* arg) {
    int id = *(int*)arg;

    sem_wait(&mutex);
    if (num_espera < CADEIRAS) {
        num_espera++;
        printf("Cliente %d sentou na sala de espera (Total esperando: %d)\n", id,
num_espera);
        sem_post(&clientes_esperando); // Acorda o barbeiro
        sem_post(&mutex);              // Libera a trava
    }
}

```

```

        sem_wait(&barbeiro_pronto); // Espera ser chamado para a cadeira de
corte
        printf("Cliente %d esta recebendo o corte.\n", id);
    } else {
        sem_post(&mutex);          // NUNCA esquecer de liberar a trava
        printf("Cliente %d encontrou a barbearia LOTADA e foi embora.\n", id);
    }
    return NULL;
}

```

```

int main() {
    pthread_t b_thread;
    pthread_t c_threads[10];
    int ids[10];

    sem_init(&mutex, 0, 1);
    sem_init(&clientes_esperando, 0, 0);
    sem_init(&barbeiro_pronto, 0, 0);

    pthread_create(&b_thread, NULL, thread_barbeiro, NULL);

    // Simula chegada sequencial de 10 clientes
    for (int i = 0; i < 10; i++) {
        ids[i] = i + 1;
        pthread_create(&c_threads[i], NULL, thread_cliente, &ids[i]);
        sleep(1);
    }

    for (int i = 0; i < 10; i++) {
        pthread_join(c_threads[i], NULL);
    }

    return 0;
}

```

5. Tabela Comparativa

Problema	Primitivas Típicas	Risco Primário	Técnica Chave
Produtor-Consumidor	2 Contadores, 1 Mutex	Deadlock (inversão)	Ordem estrita dos <code>wait()</code>
Jantar dos Filósofos	Vetor de Semáforos	Deadlock / Starvation	Quebra da Espera Circular
Barbeiro Adormecido	Mutex e Sem. Binários	Race Condition no contador	Sincronização Cliente-Servidor

Analogias Visuais:

- **Produtor:** Uma linha de montagem de caixas.
- **Barbeiro:** Uma sala de espera de consultório com apenas uma vaga no médico.
- **Filósofos:** Recursos de hardware (impressoras) sendo disputados por processos.

6. Questões

Questão 01: Inversão de Primitivas no Produtor-Consumidor (POSCOMP / Concursos)

Enunciado:

Considere o problema clássico do Produtor-Consumidor com buffer limitado de capacidade N , sincronizado por três semáforos: **mutex** (inicializado em 1), **empty** (inicializado em N) e **full** (inicializado em 0). Um desenvolvedor implementou o processo Produtor com a seguinte estrutura de código:

C

```
// PROCESSO PRODUTOR
```

```
while (true) {
```

```
    item = produzir_item();
```

```
    wait(mutex); // Operação 1
```

```
    wait(empty); // Operação 2
```

```
    inserir_no_buffer(item);
```

```
signal(mutex);  
  
signal(full);  
  
}
```

Caso o buffer atinja a sua capacidade máxima (N itens preenchidos) e o processo Produtor tente inserir um novo item executando as linhas na ordem apresentada, o comportamento observado no sistema será:

- A) Uma condição de corrida que causará sobrescrita dos itens mais antigos no buffer antes de serem lidos pelo Consumidor.
- B) A ocorrência de *Deadlock*, pois o Produtor reterá a posse da exclusão mútua (**mutex**) e ficará bloqueado em **empty**, impedindo que o Consumidor acesse a região crítica para remover itens e liberar espaço via **signal(empty)**.
- C) Execução em *livelock*, onde ambos os processos alternam continuamente o uso da CPU sem realizar trabalho útil.
- D) O sistema continuará funcionando normalmente, pois semáforos contadores possuem precedência de hardware sobre semáforos binários.
- E) A ocorrência de *Starvation* temporária exclusiva do Consumidor, sem afetar o ciclo do Produtor.

Análise das Alternativas Incorretas:

- Por que a A está incorreta? Uma condição de corrida ocorreria se a exclusão mútua fosse violada ou inexistente. Aqui, o **mutex** está trancado; o problema não é a sobrescrita de dados, mas sim o congelamento completo de avanço de ambos os processos.
- Por que a C está incorreta? No *livelock*, os processos continuam em execução ativa (gastando ciclos de clock) mudando de estado em resposta um ao outro sem avançar. Aqui, o processo entra em estado bloqueado/dormindo no semáforo **empty**; não há gasto de processamento.
- Por que a D está incorreta? Não existe "precedência de hardware" entre semáforos inteiros e binários no sistema operacional. A semântica das primitivas **wait** depende estritamente da ordem sequencial definida no código do programa.
- Por que a E está incorreta? O problema afeta ambos os processos simultaneamente, configurando um impasse mútuo irreversível (*Deadlock*), e não uma inanição unilateral (*Starvation*).

Por que a B é a Correta?

Quando o buffer está cheio, temos **empty = 0** e **mutex = 1**.

1. O Produtor executa **wait(mutex)**. Como **mutex** era 1, ele é decrementado para 0 e o Produtor entra na posse da chave da região crítica.
2. Em seguida, o Produtor executa **wait(empty)**. Como **empty = 0**, ele é decrementado para -1 e o Produtor é colocado para dormir (bloqueado) na fila do semáforo **empty**.
3. O Consumidor tenta executar o seu laço para consumir e liberar espaço. A primeira instrução após verificar **full** é **wait(mutex)**.
4. Como o Produtor dormiu segurando o **mutex**, o Consumidor encontra **mutex = 0**, decrementa para -1 e também é bloqueado na fila do **mutex**.
5. Estabelece-se a dependência circular (Condição de Coffman): o Produtor espera o Consumidor dar **signal(empty)**, mas o Consumidor não consegue chegar ao **signal(empty)** porque está travado no **mutex** detido pelo Produtor. O resultado formal é Deadlock.

Questão 02: Deadlock Simétrico e Quebra de Coffman no Jantar dos Filósofos (POSCOMP)

Enunciado:

No problema do Jantar dos Filósofos proposto por Dijkstra, cinco filósofos sentam-se à mesa circular e necessitam de dois garfos compartilhados adjacentes (o garfo à esquerda e o à direita) para comer. Se todos os cinco filósofos tentarem pegar simultaneamente o garfo à sua esquerda antes de tentar pegar o garfo à sua direita, o sistema entra em *Deadlock*. Qual das seguintes estratégias elimina formalmente a condição de **Espera Circular** (*Circular Wait*) de Coffman, impedindo o surgimento do impasse sem restringir a quantidade de cadeiras?

- **A)** Implementar uma espera ativa (*busy waiting*) com **sleep()** de tempo aleatório caso o garfo direito esteja ocupado.
- **B)** Forçar a preempção do garfo pelo filósofo que tiver a maior prioridade definida no escalonador de tempo real.
- **C)** Quebrar a simetria de aquisição: fazer com que os filósofos com índice ímpar peguem primeiro o garfo esquerdo e depois o direito, enquanto os filósofos com índice par peguem primeiro o garfo direito e depois o esquerdo.
- **D)** Substituir os 5 semáforos binários dos garfos por um único semáforo contador initialized com valor 5.
- **E)** Permitir que um filósofo pegue um garfo apenas quando estiver no estado **PENSANDO**.

Análise das Alternativas Incorretas:

- **Por que a A está incorreta?** Usar **sleep()** aleatório tenta contornar o problema probabilisticamente (como no protocolo Ethernet CSMA/CD), mas não

oferece garantia matemática formal: o sistema pode entrar em *livelock* caso os filósofos continuem pegando e soltando garfos sincronizadamente.

- **Por que a B está incorreta?** A preempção forçada de recursos alocados viola o princípio básico de consistência da exclusão mútua em estruturas não compartilháveis e quebra a condição de *Não Preempção*, não a de *Espera Circular*.
- **Por que a D está incorreta?** Um semáforo de valor 5 não resolveria a alocação dos recursos corretos. Os garfos são recursos físicos pontuais e vizinhos; um semáforo global não garante que dois filósofos adjacentes não peguem o mesmo garfo.
- **Por que a E está incorreta?** No modelo de estados (Tanenbaum), o filósofo transita de **PENSANDO** para **COM FOME** antes de tentar pegar os garfos. Ele nunca adquire garfos enquanto está pensando.

Por que a C é a Correta?

Para que ocorra *Deadlock*, uma cadeia fechada de processos deve existir: P_0 espera recurso de P_1 , P_1 de P_2 , ..., e P_4 de P_0 .

Ao introduzir a **assimetria** (filósofos pares invertem a ordem dos garfos em relação aos ímpares):

1. Considere os filósofos 0 (par) e 1 (ímpar) disputando o garfo entre eles (Garfo 1).
2. O filósofo 0 pega primeiro o garfo à **direita** (Garfo 1).
3. O filósofo 1 tenta pegar primeiro o garfo à **esquerda** (que também é o Garfo 1).
4. Ambos competirão de imediato pelo **mesmo garfo inicial**.
5. Um deles vencerá a disputa e pegará o garfo; o outro será bloqueado **antes** de conseguir reter qualquer recurso. Isso destrói a possibilidade de que todos os filósofos retenham 1 garfo ao mesmo tempo, quebrando a cadeia cíclica de dependência e inviabilizando matematicamente a **Espera Circular**.

Questão 03: Princípio da Casa dos Pombos e Limite de Capacidade (ENADE / Concursos)

Enunciado:

Uma das soluções clássicas para evitar deadlocks no Jantar dos Filósofos consiste em admitir a entrada de no máximo $N - 1$ filósofos concorrendo simultaneamente à

mesa de refeição (ou seja, 4 filósofos para 5 garfos disponíveis), o que pode ser controlado facilmente por um semáforo contador initialized com valor 4. Do ponto de vista analítico, por que essa abordagem garante que o sistema nunca entrará em *Deadlock*?

- **A)** Porque com 4 filósofos na sala, o sistema converte a execução paralela em execução serial estrita.
- **B)** Porque a exclusão mútua dos garfos deixa de ser necessária quando há menos processos do que recursos.
- **C)** Pela aplicação do Princípio da Casa dos Pombos (*Pigeonhole Principle*): havendo 5 garfos distribuídos entre no máximo 4 filósofos famintos, pelo menos um filósofo conseguirá alocar os dois garfos adjacentes necessários, comerá, liberará os recursos e permitirá o avanço dos demais.
- **D)** Porque impede que qualquer filósofo entre no estado **FOME** ao mesmo tempo que outro.
- **E)** Porque os garfos não alocados tornam-se virtuais via paginação sob demanda na memória principal.

Análise das Alternativas Incorretas:

- **Por que a A está incorreta?** O sistema continua sendo concorrente e paralelo; dois filósofos não vizinhos (por exemplo, os filósofos 0 e 2) podem comer perfeitamente ao mesmo tempo. Não há serialização estrita.
- **Por que a B está incorreta?** Cada garfo continua sendo um recurso estritamente não compartilhável. A exclusão mútua permanece obrigatória.
- **Por que a D está incorreta?** Todos os 4 filósofos presentes na sala podem sentir fome simultaneamente; a restrição apenas garante que eles não ficarão todos travados com 1 garfo cada.
- **Por que a E está incorreta?** Trata-se de uma alternativa absurda: garfos no problema clássico modelam recursos físicos de exclusão mútua (como impressoras, fitas magnéticas ou regiões de memória não reentrantes), não memória virtual paginável.

Por que a C é a Correta?

Suponha o pior cenário possível: todos os 4 filósofos autorizados a sentar decidem comer e pegam o seu primeiro garfo.

- Recursos totais disponíveis: **5 garfos**.
- Processos ativos na disputa: **4 filósofos**.
- Garfos alocados inicialmente (1 para cada): **4 garfos consumidos**.
- Garfo restante sobre a mesa: $5 - 4 = 1$ **garfo livre**.

Como o garfo livre está localizado entre dois filósofos, aquele que estiver posicionado ao lado dele terá o seu segundo garfo disponível imediatamente. Esse filósofo pegará o segundo garfo, terá ambos em mãos, completará sua refeição e executará os **signal()** dos seus dois garfos ao terminar, liberando o caminho para os

vizinhos. Como sempre há pelo menos um processo capaz de concluir sua tarefa, o bloqueio mútuo generalizado torna-se impossível.

Questão 04: Região Crítica e Tratamento de Desistência no Barbeiro Adormecido (POSCOMP)

Enunciado:

Considere o problema do Barbeiro Adormecido com 1 cadeira de corte e N cadeiras de espera na sala de espera. O semáforo binário `mutex` é utilizado para proteger a variável compartilhada `num_espera`. Analise o seguinte trecho da implementação da thread do Cliente:

C

```
// THREAD CLIENTE
```

```
wait(mutex);
```

```
if (num_espera < N) {
```

```
    num_espera++;
```

```
    signal(clientes_esperando);
```

```
    signal(mutex);
```

```
    wait(barbeiro_pronto);
```

```
    receber_corte();
```

```
} else {
```

```
    // PONTO DE SAÍDA (SALA LOTADA)
```

```
    [INSTRUÇÃO OBRIGATÓRIA]
```

```
    desistir_e_ir_embora();
```

```
}
```

Caso o programador **esqueça** de incluir a `[INSTRUÇÃO OBRIGATÓRIA]` no bloco `else` (quando o cliente encontra as cadeiras cheias e precisa ir embora), o que acontecerá com o sistema a partir desse momento?

- **A)** O cliente que desistiu cortará o cabelo mesmo sem sentar na cadeira.
- **B)** A variável `num_espera` assumirá um valor negativo devido a um estouro de pilha.
- **C)** Todos os próximos clientes que chegarem à barbearia ficarão travados eternamente na linha `wait(mutex)`, e o barbeiro nunca mais conseguirá alterar `num_espera` após terminar o corte atual, congelando o sistema.
- **D)** O sistema continuará operando normalmente, pois a instrução `else` fecha o escopo da região crítica automaticamente no sistema operacional.
- **E)** O barbeiro entrará em laço infinito cortando o cabelo do cliente ausente.

Análise das Alternativas Incorretas:

- **Por que a A está incorreta?** O cliente pula o bloco `if` e vai direto para a desistência; ele não chega a executar `receber_corte()`.
- **Por que a B está incorreta?** `num_espera` não será alterado no `else`; ele permanecerá travado no valor N.
- **Por que a D está incorreta?** Semáforos no nível de código/SO não possuem "fechamento automático de escopo". O `wait()` decrementa o semáforo; se o `signal()` correspondente não for disparado explicitamente pelo programador em todos os caminhos de execução possíveis, o recurso continuará trancado para sempre.
- **Por que a E está incorreta?** O barbeiro só atende se receber sinal em `clientes_esperando`. Como o cliente desistiu, ele não sinaliza o barbeiro.

Por que a C é a Correta?

1. Antes do `if`, o cliente que acabou de chegar executou `wait(mutex)` com sucesso, decrementando o `mutex` de 1 para 0 (trancando o acesso à região crítica).
2. Ele avalia `if (num_espera < N)`. Sendo a barbearia lotada (`num_espera == N`), a condição é falsa e o fluxo salta para o `else`.
3. Se o programador não colocar `signal(mutex);` dentro do `else`, o cliente vai embora deixando o `mutex` valendo 0.

4. Quando o próximo cliente chegar à barbearia e executar `wait(mutex)`, encontrará o valor 0, decrementará para -1 e ficará **bloqueado eternamente**.
5. Pior ainda: quando o barbeiro terminar o corte do cliente atual, ele tentará decrementar o contador de espera executando `wait(mutex)`. Como o `mutex` ficou abandonado em zero, **o próprio barbeiro também travará**. A barbearia para completamente.

A [INSTRUÇÃO OBRIGATÓRIA] é, sem dúvida, `signal(mutex);`.

7. Materiais de Apoio:

Materiais de Apoio e Referências Práticas

Bancos de Questões e Exames Oficiais

- [Repositório Awesome-POSCOMP \(GitHub\)](#)
- [Repositório ENADE Computação \(GitHub\)](#).

Problema do Jantar dos Filósofos (*Dining Philosophers*)

- [O Jantar dos Filósofos: Artigo Conceitual e Sincronização](#)
- [Implementação do Jantar dos Filósofos em Java \(DEINFO/UEPG\)](#)

Problema do Produtor-Consumidor (*Producer-Consumer*)

- [Producer-Consumer Problem in Operating Systems \(Scaler Topics\)](#)
- [Implementação do Produtor-Consumidor em Java \(DEINFO/UEPG\)](#)

Problema do Barbeiro Adormecido (*Sleeping Barber*)

- [Barbeiro Dorminhoco: Modelagem e Algoritmo \(IC/UNICAMP\)](#)
- [Implementação do Barbeiro Dorminhoco em Python \(DEINFO/UEPG\)](#)

8. Referências

- MACHADO, Francis Berenger; MAIA, Luiz Paulo. **Arquitetura de Sistemas Operacionais**. 5. ed. Rio de Janeiro: LTC, 2014.
- SILBERSCHATZ, Abraham; GALVIN, Peter Baer; GAGNE, Greg. **Fundamentos de Sistemas Operacionais**. 9. ed. Rio de Janeiro: LTC, 2015.
- TANENBAUM, Andrew S.; BOS, Herbert. **Modern Operating Systems**. 5. ed. Boston: Pearson, 2023.

